Introduction to Programming & Java Memory Architecture

Programming Paradigms, Static vs Dynamic Typing, and Stack/Heap Memory in Java

---

Introduction to Programming

Programming means giving clear, step-by-step instructions to a computer so it can perform tasks or solve problems. We write these instructions using programming languages, which follow different styles or methods called **programming paradigms**.

Types of Languages (Programming Paradigms)

1. Procedural Programming

Explanation: A programming style where code is written as a list of step-by-step instructions or functions that directly update and change data.

Real-World Example: Making a cup of tea. You boil water, add tea leaves, and add sugar step-by-step. The state of the ingredients changes directly in the pot.

JAVA

```
public class ProceduralExample {
    public static void main(String[] args) {
        int balance = 1000;
        balance = deposit(balance, 500);
        System.out.println(balance);
    }

    public static int deposit(int currentBalance, int amount) {
        return currentBalance + amount;
    }
}
```

Explanation: The `deposit` function takes the number, performs the addition step, and updates the balance variable.

2. Functional Programming

Explanation: A programming style where programs are built using mathematical functions. Data is never modified directly; functions take an input and return a brand-new output while keeping the original input safe.

Real-World Example: Taking a photo of a document. If you add a black-and-white filter to the photo on your phone, the photo changes, but the original paper document in your hand remains untouched.

JAVA

```
public class FunctionalExample {
    public static void main(String[] args) {
        int originalMarks = 50;
        int newMarks = addGraceMarks(originalMarks);
```

```
        System.out.println(originalMarks);
        System.out.println(newMarks);
    }

    public static int addGraceMarks(int marks) {
        return marks + 5;
    }
}
```

Explanation: The `addGraceMarks` function creates and returns 55 as a new value without modifying `originalMarks`, which stays 50.

3. Object-Oriented Programming (OOP)

Explanation: A programming style where code is organized into “Objects”. An object bundles data (properties) and actions (methods) together into a single unit.

Real-World Example: A Fan. It has properties like speed and color, and it has actions like turnOn() and changeSpeed().

JAVA

```
class Fan {
    int speed = 0;

    void turnOn() {
        speed = 1;
    }
}

public class OOPExample {
    public static void main(String[] args) {
        Fan myFan = new Fan();
        myFan.turnOn();
        System.out.println(myFan.speed);
    }
}
```

Explanation: `myFan` is an object that controls its own internal variable `speed` through its own function `turnOn()`.

Static vs Dynamic Language

Statically-Typed: Variable types are checked before running (at compile time). You must declare the data type explicitly, and you cannot assign a different type later.

Dynamically-Typed: Variable types are checked while running (at runtime). You do not declare types, and the same variable can hold numbers, text, or lists at different points in the program.

Comparison Table

Feature	Statically-Typed (Java)	Dynamically-Typed (Python)
When Types are Checked	Compile-time (before code runs)	Runtime (while code runs)
Type Declaration	Mandatory (int, String, etc.)	Not required
Type Flexibility	Cannot change variable type	Can change variable type freely

Real-World Example — Static Typing: A square shape sorter toy. Only a square block fits into the square hole. If you try to force a circle in, it gets stopped immediately before you even start playing.

Real-World Example — Dynamic Typing: An open cardboard box. You can put a toy car inside now, remove it, and place a book inside later without changing the box.

JAVA — STATICALLY-TYPED

```
public class StaticExample {
    public static void main(String[] args) {
        int score = 100;
        // score = "Hundred"; // Compile-time error: Java will not allow this
        System.out.println(score);
    }
}
```

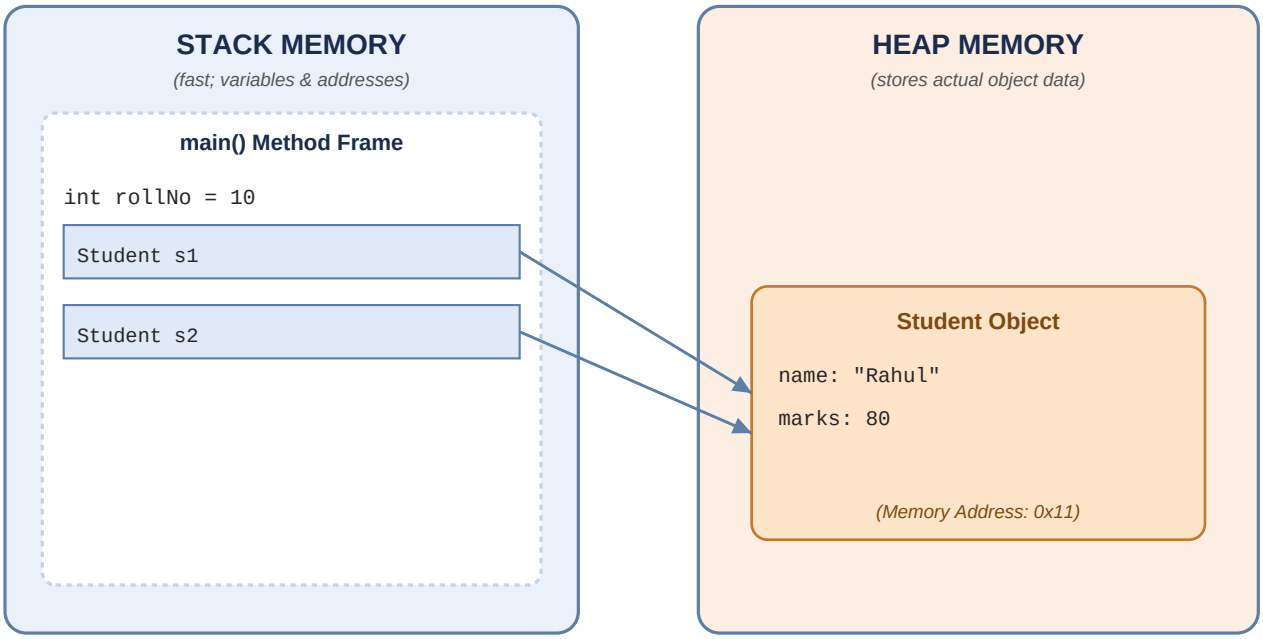
PYTHON — DYNAMICALLY-TYPED

```
score = 100
score = "Hundred"  # Valid in Python: adapts its type at runtime
print(score)
```

Stack and Heap Memory Management

When a Java program executes, the Java Virtual Machine (JVM) divides the computer's memory (RAM) into two major areas: **Stack** and **Heap**.

Memory Architecture Diagram



Both `s1` and `s2` are separate reference variables on the Stack, but they store the same Heap address (0x11) — so they both point to the same object.

1. Stack Memory

What is it? Fast, temporary storage managed by CPU execution in a Last-In, First-Out (LIFO) order.

What is stored here?

Stored on the Stack	Example
Primitive variables	int, float, boolean, char
Reference variables	variables holding memory addresses that point to objects on the Heap

Lifecycle: When a method finishes executing, its stack memory frame is removed immediately.

2. Heap Memory

What is it? A large, shared memory pool used across the entire application.

What is stored here? Actual objects and arrays created using the `new` keyword.

Lifecycle: Objects stay in the Heap as long as any active reference variable points to them. When no reference points to an object, Java's Garbage Collector cleans it up automatically.

3. Reference Variables vs. Actual Data

Real-World Example: Heap Object: a physical house built on a street. Reference Variable: a piece of paper in your pocket with the address of that house written on it. If two people have slips of paper with the exact same address, both walk to and interact with the same physical house.

JAVA

```
class Student {
    String name = "Rahul";
    int marks = 80;
}

public class MemoryDemo {
    public static void main(String[] args) {
        int rollNo = 10;

        Student s1 = new Student();
        Student s2 = s1;

        s2.marks = 95;

        System.out.println(s1.marks);
    }
}
```

Step-by-Step Memory Execution

Code	What Happens
`int rollNo = 10;`	rollNo is a primitive type. The value 10 is stored directly on the Stack.
`Student s1 = new Student();`	new Student() allocates memory on the Heap (e.g. at address 0x11) to hold name = "Rahul" and marks = 80. s1 is created on the Stack and stores the address 0x11.
`Student s2 = s1;`	s2 is created on the Stack. It copies the address 0x11 from s1. Both s1 and s2 now point to the same object on the Heap.
`s2.marks = 95;`	Using s2, we go to address 0x11 on the Heap and update marks to 95.
`System.out.println(s1.marks);`	Prints 95 because s1 and s2 reference the exact same object.